

---

# 1. Introdução ao Matlab

*Um computador é uma ferramenta essencial na resolução de problemas de Engenharia. Nestes apontamentos faz-se uma introdução ao Matlab, um pacote de software com a capacidade de computação numérica, análise de dados e gráficos.*

*O software Matlab foi originalmente desenvolvido para um uso específico e daí a origem do seu nome “Matrix Laboratory”. Actualmente as suas capacidades vão muito além das originalmente pensadas: é um sistema interactivo e permite todas as capacidades de uma linguagem de programação. Como propriedade fundamental o facto de que o Matlab faz uso de matrizes como estrutura de dados básica.*

*Depois de inicializar uma sessão em Matlab é visualizado o sinal de prompt (>>) que significa que o Matlab está pronto a receber um comando do utilizador.*

---

## 1.1 Tipo de Dados Fundamentais

O Matlab trabalha fundamentalmente com um tipo de dados: matrizes de números reais ou complexos. Casos especiais de matrizes são escalares (matriz  $1 \times 1$ ) e vectores (matriz linha  $1 \times n$  ou matriz coluna  $n \times 1$ ).

### 1.1.1 Matrizes

Quando se pretende resolver um problema de engenharia é importante ter em conta a forma como vão ser visualizados os dados relativos ao problema em questão. Por vezes os dados são um único número, como por exemplo o raio de um círculo. Noutros casos o conjunto de dados pode ser a coordenada de um ponto no plano e tem-se um conjunto de dados constituído por um par de números  $(x,y)$ . Noutros ainda podemos ter um conjunto de dados da forma  $(x,y,z)$ .

Contudo, quaisquer que sejam os dados, é sempre possível representá-los usando a notação matricial. Uma matriz não é senão um conjunto de números ordenados em linhas ou colunas.

## 1.1 Tipos de dados fundamentais

---

Assim um único ponto  $x$  pode ser considerado uma matriz com uma linha e uma coluna,

$x$  - matriz (1x1)

Um ponto  $(x,y)$  como uma matriz de uma linha e duas colunas.

$[x, y]$  - matriz (1x2)

### 1.1.2 Inicialização de matrizes

Vamos agora ver como podem ser definidas e inicializadas matrizes no Matlab. Serão apresentados quatro métodos para o fazer: de uma forma explícita, como resultado de uma operação, como resultado da leitura de um ficheiro de dados e introduzida directamente pelo utilizador. Para os exemplos que se irão apresentar consideram-se as seguintes matrizes:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \quad B = \begin{bmatrix} -1 & -2 \\ -3 & -4 \\ -5 & -6 \end{bmatrix}$$

#### De uma forma explícita

A forma mais simples de definir uma matriz é usar uma lista de números entre parênteses rectos. Assim a matriz  $A$  pode ser definida como,



```
>> A = [ 1 2 3 ; 4 5 6 ]
```

O nome de uma matriz deverá começar por uma letra e conter até um máximo de 19 caracteres. Podem ser utilizadas vírgulas ou espaços em branco para separar colunas e para separar linhas devem ser utilizados pontos e vírgulas ou mudanças de linha. Outra alternativa à definição da matriz  $A$  poderia ser:



```
>> A = [ 1,2,3  
4,5,6]
```

#### Como resultado de alguma operação

É também possível definir uma matriz como o resultado de alguma operação. Por exemplo se desejar conhecer o dobro da matriz  $A$ , então pode-se definir uma nova matriz  $C$ .

## 1.1 Tipos de dados fundamentais



```
>> C = 2*A
```

### A partir de um ficheiro de dados

Outra forma de definir matrizes é a partir de informação previamente armazenada em ficheiros de dados. É possível lidar com dois tipos de ficheiros em Matlab: *MAT-files* e *ASCII*. No primeiro caso os dados são armazenados segundo um formato binário; no segundo caso os dados são constituídos por caracteres ASCII.

Os ficheiros MAT são gerados no Matlab a partir do comando **Save**, que deve conter o nome da matriz a guardar e o respectivo nome do ficheiro. Automaticamente o Matlab atribui-lhe a extensão *.MAT*.

Por exemplo o comando



```
>> save dados B
```

permite armazenar num ficheiro de nome dados a matriz B. Assim é possível, mesmo numa futura sessão, aceder novamente ao valor de B, tendo que para isso executar o comando **Load**.



```
>> load dados
```

Automaticamente o Matlab permite definir a matriz B.

Se forem utilizados ficheiros ASCII então os valores a armazenar terão obrigatoriamente de conter apenas informação numérica possível de ser facilmente gerada com um qualquer vulgar editor ou processador de texto. Outra alternativa é gerar o ficheiro à custa do Matlab usando o comando **save** e a opção */ascii*.



```
>> save dados1.dat B -ascii
```

O resultado da operação anterior seria a criação de um ficheiro de texto com a seguinte informação:



|          |          |
|----------|----------|
| -1.00000 | -2.00000 |
| -3.00000 | -4.00000 |
| -5.00000 | -6.00000 |

## 1.1 Tipos de dados fundamentais

---

Tal como no caso anterior é possível aceder à posteriormente a esta informação usando o comando **load**.



```
>> load dados1.dat
```

Contudo neste caso o nome da matriz que se definia seria dados1 e não B, portanto o nome do ficheiro em causa e não a variável que aí foi armazenada.

### Introdução de Dados pelo Utilizador. Operador (;)

Os valores de uma matriz podem ser introduzidos através do teclado directamente pelo utilizador usando o comando **input**. Por exemplo a seguinte instrução:



```
>> numero = input(' Valor do numero ')
```

permite ao utilizador definir um dado valor para a variável numero. Se o comando não for finalizado por (;) o valor de numero será mostrado no écran, caso contrário o valor de numero é apenas armazenado na variável e não é mostrado.



```
>> numero = input(' Valor do valor de numero ´) ;
```

Este uso do operador (;) no final de cada instrução é válido para qualquer comando Matlab.

### 1.1.3 Referência a elementos de matrizes

Uma vez que uma matriz é um conjunto de valores organizados segundo linhas e colunas deve ser possível aceder a um dos seus elementos ou mesmo a um subconjunto dos seus elementos.

#### Um elemento (escalar)

Para aceder a um elemento da matriz deve-se indicar qual a linha e a coluna. Assim,



```
>> a=A(2,3)
```

permite aceder ao elemento localizado na segunda linha e terceira coluna da matriz A, portanto a=6. De uma forma geral pode-se escrever:



**$A(i,j)$  - elemento situado na linha  $i$  e coluna  $j$**

### Subconjunto de elementos, operador (:)

Para aceder a um subconjunto de elementos é usado o operador dois pontos (:). Eis alguns exemplos que permitem compreender o seu funcionamento:



```
>> C= A(1:2, 2:3)
```

Significa que se pretendem os elementos da matriz  $A$ , desde a primeira à segunda linha e desde a segunda até à terceira coluna. O resultado seria portanto

$$C = \begin{bmatrix} 2 & 3 \\ 5 & 6 \end{bmatrix}$$

De uma forma geral pode-se escrever:



**$A(i_{\min} : i_{\max}, j_{\min} : j_{\max})$**

O resultado obtido são todos os elementos da matriz  $A$  definidos pelas linhas  $i_{\min}$  até  $i_{\max}$  e pelas colunas  $j_{\min}$  até  $j_{\max}$ . É possível também aceder a todos os elementos de uma linha/coluna mesmo que não se conheça a sua dimensão.

Por exemplo:



```
>> C = A(1:1, :)
```

define a matriz  $C$  como sendo

$$C = [1 \quad 2 \quad 3]$$

ou seja definidos pela primeira linha e todas as colunas. Uma expressão equivalente à anterior seria:



```
>> C = A(1, :)
```

No seguinte exemplo define-se a matriz  $C$  como sendo igual à  $A$ .

## 1.1 Tipos de dados fundamentais

---



```
>> C = A(:, :)
```

O operador (:) permite em Matlab definir um vector da seguinte forma:



```
>> valor=inicio:incremento:fim
```

O resultado é um vector cujo primeiro valor é igual a inicio e os restantes serão incrementados com um valor igual a incremento até ao último valor que será igual a fim.

Por exemplo executar o seguinte comando



```
>> tempo=0:2:10
```

teria o mesmo resultado que:



```
>> tempo = [ 0 2 4 6 8 10 ]
```

### 1.1.4 Formatos e visualização numérica

Existem várias formas de visualizar o conteúdo de uma matriz. A mais simples é digitar o nome da matriz e automaticamente o seu valor será mostrado. Existem vários formatos pré-definidos em Matlab. Referem-se quatro:



```
format short
```

```
format long
```

```
format short e
```

```
format long e
```

Os dois primeiros utilizam uma formatação corrente e os dois últimos científica. Para perceber as diferenças entre eles apresenta-se o seguinte exemplo: admite-se que existe uma variável com o valor de 1/7. A sua representação em cada um dos formatos referidos seria:

short: 0.1429

long: 0.14285714285714

short e: 1.4286e-001

long e: 1.428571428571428e-001

### 1.1.5 Visualização gráfica

Suponhamos agora que se pretende fazer um gráfico com os valores da matriz em vez de, como na secção anterior, fazer apenas a sua visualização numérica. Na secção 1.5 este assunto será tratado com pormenor, por agora apresenta-se apenas uma breve introdução indicando o modo de visualizar um gráfico (x,y) a partir de dados armazenados em dois vectores.

| Horas | Temperatura |
|-------|-------------|
| 0     | 9           |
| 2     | 8           |
| 4     | 6           |
| 6     | 5           |
| 8     | 8           |
| 10    | 10          |
| 12    | 14          |
| 14    | 17          |
| 16    | 15          |
| 18    | 13          |
| 20    | 11          |
| 22    | 10          |

**Tabela 1.1:** Valores da temperatura do ar.

Consideremos que foram observados os valores da temperatura do ar num determinado local durante as 24 horas do dia, de duas em duas horas. Os valores recolhidos mostram-se na tabela 1.1.

Assumindo que os valores foram guardados em dois vectores,



```
>> x=[ 0: 2: 22]
```

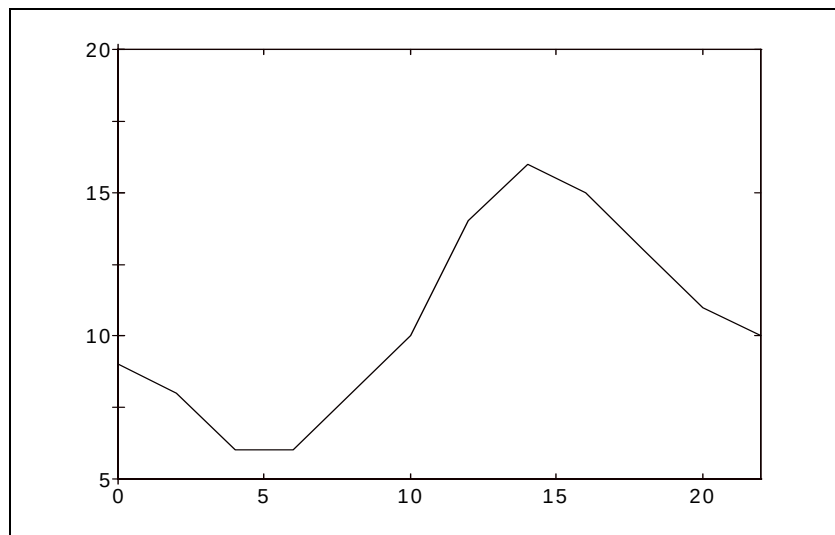
```
>> y=[ 9 8 6 5 8 10 14 17 15 13 11 10]
```

então o comando,



```
>> plot(x,y)
```

permite visualizar os dados da seguinte forma:



**Figura 1.1:** Gráfico x-y.

## 1.2 Operações Elementares

As operações de adição, subtração, multiplicação e divisão são consideradas as operações fundamentais. Nesta secção além destas mostra-se como podem ser efectuadas outras operações tais como, raiz quadrada, logaritmos, etc., aplicadas a escalares, vectores e matrizes. Serão também definidas formas de tratar números complexos.

### 1.2.1 Valores escalares especiais

O Matlab dispõe de um número de valores pré-definidos:

|        |                                                                                               |
|--------|-----------------------------------------------------------------------------------------------|
| • ans  | Variável usada para guardar o valor da última operação efectuada                              |
| • pi   | Valor de $\pi=3.1415...$                                                                      |
| • i, j | $\sqrt{-1}$ - Utilizado em números complexos                                                  |
| • Inf  | $\infty$ - Valor de infinito                                                                  |
| • Nan  | Significa que não é um número                                                                 |
| • eps  | Precisão do computador usado, ou seja, é a diferença entre 1.0 e o valor decimal mais próximo |

### 1.2.2 Matrizes especiais

Tal como no caso de escalares o Matlab dispõe de matrizes com valores especiais. Aquelas com mais interesse enumeram-se de seguida:

#### Zeros (*Zeros*)

Permite gerar uma matriz com todos os elementos nulos. Se o argumento for apenas um escalar, por exemplo,



```
>> A= zeros(6)
```

então a matriz *A* gerada é uma matriz quadrada, neste caso de 6 linhas e 6 colunas. Podem, no caso mais genérico, ser definidos dois parâmetros. Por exemplo,



```
>> A= zeros(2,3)
```

permite gerar uma matriz *A* de 2 linhas e 3 colunas em que todos os elementos são nulos.

#### Uns (*Ones*)

A função **ones** é idêntica à *zeros*, diferindo apenas no facto de que a matriz gerada não é constituída por valores nulos mas todos os elementos são iguais a 1. Assim, por exemplo,



```
>> A= ones(1,1)
```

permite gerar uma matriz (escalar) *A*=1.

#### Identidade (*Eye*)

Esta função permite criar uma matriz identidade. Uma matriz identidade é uma matriz quadrada de valores nulos com excepção dos elementos da diagonal que são unitários. Um exemplo:



```
>> A= eye(3)
```

permite gerar a seguinte matriz:

$$A = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

### 1.2.3 Operações com Escalares

As operações elementares possíveis de efectuar com escalares mostram-se na tabela seguinte

| Operação           | Forma Algébrica | Matlab          |
|--------------------|-----------------|-----------------|
| Adição             | $a + b$         | $a + b$         |
| Subtracção         | $a - b$         | $a - b$         |
| Multiplicação      | $a * b$         | $a * b$         |
| Divisão à direita  | $\frac{a}{b}$   | $a / b$         |
| Divisão à esquerda | $\frac{b}{a}$   | $a \setminus b$ |
| Exponencial        | $a^b$           | $a^b$           |

**Tabela 1.2:** Operações com escalares.

### 1.2.4 Operações elemento a elemento: vectores

Admitamos que dispomos de dois vectores:



```
>> A = [ 2 5 6 ]
```

```
>> B = [ 2 3 5 ]
```

Deseja-se definir um terceiro vector que seja o resultado da multiplicação elemento a elemento deste dois vectores. Uma forma seria efectuar repetidamente:



```
>> C(1) = A(1)*B(1)
```

```
>> C(2) = A(2)*B(2)
```

```
>> C(3) = A(3)*B(3)
```

Evidentemente tal processo seria inviável para vectores de maiores dimensões. Utilizando o Matlab tal operação pode ser facilmente efectuada usando o seguinte comando:

## 1.2 Operações elementares



```
>> C = A.*B
```

O ponto final (.) antes do sinal de multiplicação (\*) representa portanto uma operação elemento a elemento. Refira-se que se este fosse omitido estaria a fazer uma multiplicação normal entre duas matrizes. Uma vez explicado o que se entende por operação elemento a elemento mostra-se na tabela seguinte as operações possíveis em Matlab.

| Operação           | Forma Algébrica | Matlab           |
|--------------------|-----------------|------------------|
| Adição             | $a + b$         | $a + b$          |
| Subtracção         | $a - b$         | $a - b$          |
| Multiplicação      | $a * b$         | $a.*b$           |
| Divisão à direita  | $\frac{a}{b}$   | $a./b$           |
| Divisão à esquerda | $\frac{b}{a}$   | $a.\backslash b$ |
| Exponencial        | $a^b$           | $a.^b$           |

**Tabela 1.3:** Operações com vectores.

Para melhor entender este tipo de operações mostram-se alguns exemplo considerando os vectores A e B acima definidos.



```
>> C = A.*B
```

O resultado é  $C = [ 4 \ 15 \ 30 ]$



```
>> C = A./B
```

O resultado é  $C = [ 1 \ 1.667 \ 1.2 ]$



```
>> C = A.\B
```

O resultado é  $C = [ 1 \ 0.6 \ 0.833 ]$



```
>> C = A.^2
```

O resultado é  $C = [ 4 \ 25 \ 36 ]$



```
>> C = (3).^A
```

O resultado é  $C = [ 9 \ 243 \ 729 ]$

Neste último caso note que se o comando fosse  $3.^A$  seria gerada uma mensagem de erro!!



```
>> C = A.^B
```

O resultado é  $C = [4 \ 125 \ 7776]$

### 1.2.5 Operações mais comuns

Além das operações comuns adição, subtração, multiplicação e divisão, é por vezes necessário outro tipo de operações com os dados. O Matlab permite uma grande variedade de funções das quais se destacam:

|                         |                                                                    |
|-------------------------|--------------------------------------------------------------------|
| • <code>abs(x)</code>   | Valor absoluto de x                                                |
| • <code>sqrt(x)</code>  | Raiz quadrada de x                                                 |
| • <code>round(x)</code> | Converte x para o valor inteiro mais perto                         |
| • <code>fix(x)</code>   | Converte x para o valor inteiro mais perto em direcção a zero      |
| • <code>floor(x)</code> | Converte x para o valor inteiro mais perto em direcção a $-\infty$ |
| • <code>ceil(x)</code>  | Converte x para o valor inteiro mais perto em direcção a $\infty$  |
| • <code>sign(x)</code>  | Devolve -1 se $x < 0$ ou 1 se $x \geq 0$                           |
| • <code>rem(x,y)</code> | Resto da divisão de x/y                                            |
| • <code>exp(x)</code>   | Exponencial de x $e^x$                                             |
| • <code>log(x)</code>   | Logaritmo natural de x na base e: $\ln x$                          |
| • <code>log10(x)</code> | Logaritmo de x na base 10: $\log_{10} x$                           |

Note-se que x pode ser um escalar, vector ou matriz. Caso seja por exemplo uma matriz a operação seno é aplicada a cada um dos elementos da matriz. Além disso, para qualquer uma das funções assume-se que o valor de x se encontra na gama permitida, caso contrário será gerado um erro.

### 1.2.6 Funções trigonométricas

|                        |                                                                                  |
|------------------------|----------------------------------------------------------------------------------|
| • <code>sin(x)</code>  | Seno do ângulo x definido em radianos                                            |
| • <code>cos(x)</code>  | Coseno do ângulo x definido em radianos                                          |
| • <code>tan(x)</code>  | Tangente do ângulo x definido em radianos                                        |
| • <code>asin(x)</code> | Arco seno ou inverso do seno de $x \in [-1, 1]$                                  |
|                        | A função devolve um ângulo compreendido entre $-\frac{\pi}{2}$ e $\frac{\pi}{2}$ |

## 1.2 Operações elementares

|                     |                                                                                                                                                          |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| • $\text{acos}(x)$  | Arco coseno ou inverso do coseno de $x \in [-1, \dots, 1]$ .<br><br>A função devolve um ângulo compreendido entre 0 e $\pi$                              |
| • $\text{atan}(x)$  | Arco tangente ou inversa da tangente de $x \in [-1, \dots, 1]$ .<br><br>A função devolve um ângulo compreendido entre $-\frac{\pi}{2}$ e $\frac{\pi}{2}$ |
| • $\text{atan2}(x)$ | Arco tangente ou inversa da tangente de $x \in [-1, \dots, 1]$ .<br><br>A função devolve um ângulo compreendido entre $-\pi$ e $\pi$                     |

### 1.2.7 Números complexos

Consideremos por exemplo a seguinte equação de segunda ordem:

$$f(x) = x^2 + 3x + 3$$

Admitindo que se deseja calcular os zeros, usando a formula resolvente

$$x_1 = \frac{-3 + \sqrt{-3}}{2} = -1.5 + 0.87\sqrt{-1} \quad x_2 = \frac{-3 - \sqrt{-3}}{2} = -1.5 - 0.87\sqrt{-1}$$

De forma a que estas raízes tenham sentido surge a definição de números complexos e também de  $\sqrt{-1}$ . Um número complexo é um número definido por  $a + bi$  tal que  $a$  e  $b$  são números reais e  $i = \sqrt{-1}$ .

#### Operações Aritméticas com Números Complexos

Dados dois números complexos:  $c_1 = a_1 + ib_1$  e  $c_2 = a_2 + ib_2$  então,

| Operação          | Resultado                                                                                                           |
|-------------------|---------------------------------------------------------------------------------------------------------------------|
| $c_1 + c_2$       | $(a_1 + a_2) + i(b_1 + b_2)$                                                                                        |
| $c_1 - c_2$       | $(a_1 - a_2) + i(b_1 - b_2)$                                                                                        |
| $c_1 * c_2$       | $(a_1 a_2 - b_1 b_2) + i(a_1 b_2 + a_2 b_1)$                                                                        |
| $\frac{c_1}{c_2}$ | $\left( \frac{a_1 a_2 + b_1 b_2}{a_2^2 + b_2^2} \right) + i \left( \frac{a_2 b_1 - b_2 a_1}{a_2^2 + b_2^2} \right)$ |
| $ c_1 $           | $\sqrt{a_1^2 + b_1^2}$                                                                                              |
| $c_1^*$           | $a_1 - ib_1$                                                                                                        |

**Tabela 1.4:** Operações com números complexos.

## 1.3 Definição de Funções pelo Utilizador: M files

Seria bastante limitado se apenas fosse possível executar as funções pré-definidas pelo Matlab. Quase sempre surge a necessidade de criar novas funções. Vai-se ver agora como é que isso é possível usando o Matlab.

### Ficheiros \*.m - M Files

Antes de explicar o conceito de função comecemos por descrever os ficheiros \*.m utilizados pelo Matlab. Até agora tem-se falado em comandos isolados como sendo a única hipótese de “comunicar” com o Matlab. Supondo que se deseja somar três números  $x$ ,  $y$  e  $z$ , usando os comandos do Matlab, ter-se-ia:



```
>> x=2
```

```
>> y=3
```

```
>> z=1
```

```
>> c= x+y+z
```

Neste caso são necessários apenas quatro comandos e facilmente se digitam os quatro comandos. O problema põe-se se for necessário digitar por exemplo 100 comandos para resolver um certo problema ... Será que é sempre necessário digitar todos os comandos ?

Certamente que não. O Matlab recorre ao uso de ficheiros \*.m para resolver o problema.

### 1.3.1 Scripts

Um ficheiro \*.m não é senão um vulgar ficheiro de texto onde são armazenados os comandos a serem executados pelo Matlab, um em cada linha e pela sequência desejada. Por exemplo, para somar os tais números  $x$ ,  $y$  e  $z$ , poder-se-ia criar um vulgar ficheiro ASCII com um qualquer editor de texto que contivesse o seguinte texto.



```
x = 2
```

```
y = 3
```

```
z = 1
```

```
c = x+y+z
```

O ficheiro poderia ter qualquer nome apenas tendo como restrição a extensão que teria de ser .m (daí o nome de ficheiros \*.m). Note-se que é importante a extensão!! Um possível nome para o ficheiro de texto poderia ser *soma.m*.

### 1.3 Definição de funções pelo utilizador: M files

---

Uma vez criado este ficheiro *soma.m* bastaria agora digitar o seguinte comando na linha de comando do Matlab (não é necessário a extensão).



```
>> soma
```

Automaticamente o Matlab executa cada uma das linhas do ficheiro *soma.m*. A este tipo de ficheiros em que os comandos são executados sequencialmente dá-se o nome de *Scripts* para os diferenciar das funções, como se verá mais adiante.

Assim sempre que se executa uma sessão em Matlab deve-se de preferência criar um ficheiro de texto e escrever aí todos os comandos. Sempre que se queira executar todas as instruções basta digitar um comando no Matlab. Doutra forma seria necessária digitar todas as instruções como comandos de Matlab.

Além disso, se for necessário modificar algum dado (por exemplo alterar o valor do número *x* de 2 para 4) basta modificar o ficheiro de texto. No modo de comando seria necessário, além de modificar o valor de *x*, digitar todas as restantes instruções.

#### 1.3.2 Funções

Enquanto que um ficheiro *\*.m* executa sequencialmente todas as instruções uma função é um ficheiro *\*.m* especial que devolve um ou mais valores. Distingue-se de um *script* principalmente pela primeira linha, que é da forma:



```
function [parâmetros saída] = nome_função (parâmetros entrada)
```

Suponha-se por exemplo que se implementa uma função capaz de somar três números *x*, *y* e *z*. Para resolver o problema começa-se por criar um ficheiro com o nome de, por exemplo, *soma.m*. Uma possível solução para o conteúdo deste ficheiro será:



```
function r = soma( x, y, z)
```

```
    r = x + y + z
```

```
end
```

Ou seja, como parâmetros de entrada tem-se os valores de *x*, *y* e *z*. Como parâmetro de saída temos o valor *r* que é a soma dos três valores. Em Matlab aceder-se-ia a esta função simplesmente fazendo:

### 1.3 Definição de funções pelo utilizador: M files



```
>> r = soma( 1, 2 ,3)
```

e o resultado seria  $r = 6$ .

Suponhamos que se deseja uma função que calcule a soma e o produto dos três elementos  $x$ ,  $y$  e  $z$ . Da mesma forma cria-se um ficheiro de nome *somaprod.m* com o seguinte conteúdo:



```
function [ soma, produto] = soma_produto( x, y, z)
    soma = x + y + z ;
    produto = x * y * z ;
end
```

Para usar esta função em Matlab executa-se,



```
>> [s,p] = somaprod( 2, 2, 3)
```

resultando  $s = 7$  e  $p = 12$ .

**Nota importante:** O Matlab identifica a função pelo nome do ficheiro (neste caso *somaprod*) e não pelo nome que lhe atribui dentro do ficheiro de texto (neste caso *soma\_produto*).

## 1.4 Controlo de Fluxo

Se por vezes uma série de instruções executada de uma forma sequencial permite resolver um determinado problema, noutros casos existe a necessidade de se executar apenas parte dos comandos em função de uma determinada condição. Pode por exemplo existir a necessidade de repetir o mesmo comando um grande número de vezes, etc. Qualquer destas situações é possível de implementar em Matlab e designam-se por operações de controlo de fluxo.

### 1.4.1 Operadores relacionais

Em Matlab existem seis operadores que permitem comparar duas matrizes de igual dimensão:

| Operador | Interpretação         |
|----------|-----------------------|
| <        | Menor do que          |
| <=       | Menor ou igual do que |
| >        | Maior do que          |

## 1.4 Controlo de fluxo

|                    |                       |
|--------------------|-----------------------|
| <code>&gt;=</code> | Maior ou igual do que |
| <code>==</code>    | Igual                 |
| <code>~=</code>    | Diferente             |

**Tabela 1.5:** Operadores relacionais.

O resultado de uma destas operações é igual a um número inteiro 1 se a condição é verdadeira e 0 se é falsa. Por exemplo, considerando a e b escalares, o comando



```
>> a < b
```

será igual a 1 se a menor do que b ou zero se isso não acontecer.

No caso de matrizes os operadores são aplicados elemento a elemento. Consideremos os seguinte vectores:



```
>> a = [ 2 4 6 ]
```

```
>> b = [ 3 5 1 ]
```

```
>> a < b
```

```
>> ans = [ 1 1 0 ]
```

O resultado da operação `a<b` é portanto verdadeira para os dois primeiros elementos e falsa para o terceiro.

### 1.4.2 Operadores lógicos

É possível combinar duas expressões lógicas usando para isso operadores lógicos e (***and***), ou (***or***) ou não (***not***).

| Operador Lógico | Símbolo            |
|-----------------|--------------------|
| and             | <code>&amp;</code> |
| or              | <code> </code>     |
| not             | <code>~</code>     |

**Tabela 1.6:** Operações lógicas.

## 1.4 Controlo de fluxo

Os operadores lógicos são possíveis de aplicar a matrizes de zeros e uns. Por exemplo a expressão,



```
>> a<b & b<c
```

é válida se cada uma das matrizes resultantes (da operação  $a<b$  e  $b<c$ ) tiverem a mesma dimensão.

### 1.4.3 Condição If

A sintaxe para o comando **if** é a seguinte:



```
if expressão lógica
```

```
    instruções
```

```
end
```

Significa que se a expressão lógica for verdadeira então o conjunto de instruções é executado; se a expressão lógica for falsa o programa “salta” o conjunto de instruções. Consideremos o seguinte exemplo:



```
if a<50
```

```
    soma=soma+a
```

```
end
```

Assumindo que  $a$  é um escalar, se  $a<50$  então o valor da soma é incrementado de  $a$  unidades; caso  $a\geq 50$  não faz nada.

É também possível definir condições **if** dentro de condições **if**, por exemplo:



```
if expressão lógica 1
```

```
    instruções 1
```

```
        if expressão lógica 2
```

```
            instruções 2
```

```
        end
```

```
    instruções 3
```

|     |
|-----|
| end |
|-----|

Se a expressão lógica 1 é verdadeira são executadas as instruções 1 e 3. Se for falsa nada é executado. Se a expressão lógica 2 é verdadeira é executado o conjunto de instruções 2.

#### 1.4.4 Condição If Else

Adicionado ao comando **if** o comando **else** permite optar por um conjunto de instruções se uma expressão lógica for verdadeira ou por outro conjunto de instruções se a expressão lógica for falsa.



|                     |
|---------------------|
| if expressão lógica |
| instruções 1        |
| else                |
| instruções 2        |
| end                 |

#### 1.4.5 Condição Elseif

Quando é necessário implementar vários níveis de condições **if-else** é preferível utilizar a condição **elseif**



|                           |
|---------------------------|
| if expressão lógica 1     |
| instruções 1              |
| elseif expressão lógica 2 |
| instruções 2              |
| elseif expressão lógica 3 |
| instruções 3              |
| end                       |

Se a expressão lógica 1 é verdadeira apenas o conjunto de instruções 1 é executado, se a expressão lógica 1 for falsa e a expressão lógica 2 é verdadeira apenas o conjunto de instruções 2 é executado e se a expressão lógica 1 e 2 forem falsas e a expressão lógica 3 for verdadeira

apenas o conjunto de instruções 3 é executado. Claro que também é possível obter o mesmo resultado apenas com instruções **if**, mas como se percebe, com um maior grau de complexidade.

### 1.4.6 Funções lógicas

O Matlab fornece um conjunto de funções lógicas úteis em condições **if**. Eis algumas delas:

|                     |                                                                                                              |
|---------------------|--------------------------------------------------------------------------------------------------------------|
| • <b>any(x)</b>     | Para cada coluna da matriz x esta função devolve 1 se algum valor é não nulo e 0 em caso contrário           |
| • <b>all(x)</b>     | Para cada coluna da matriz x esta função devolve 1 se todos os valores forem não nulos e 0 em caso contrário |
| • <b>isnan(x)</b>   | Devolve uma matriz com uns nos elementos de valor infinito e zero se não o forem                             |
| • <b>isempty(x)</b> | Devolve 1 se a matriz é uma matriz vazia, 0 em caso contrário                                                |

Seguem-se alguns exemplos. Seja a matriz A definida por:

$$A = \begin{bmatrix} 0 & 2 & 0 & 4 \\ 0 & 1 & 0 & 6 \\ 0 & 5 & 6 & 7 \end{bmatrix}$$



```
>> any(A)
```

O resultado será [0 1 1 1]



```
>> all(A)
```

O resultado será [0 1 0 1]



```
>> isnan(A)
```

O resultado será  $\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$



```
>> isempty(A)
```

O resultado será 0.

### 1.4.7 Ciclo de repetição For

Em Matlab existem duas formas possíveis de executar ciclos: os comandos **for** e **while**. Nesta secção apresenta-se o ciclo **for** e na secção seguinte o ciclo **While**. A diferença principal entre ambos é a seguinte: no ciclo **for** o número de repetições do ciclo é perfeitamente conhecida, no **while** esse número desconhece-se à partida. A sintaxe do comando **for** é a seguinte:



```
for index=expressão
```

```
    instruções
```

```
end
```

Suponha que dispõe de 10 números armazenados num vector A e que pretende contar quais são positivos e quais são negativos. Uma possível solução seria:



```
positivo = 0;
```

```
negativo = 0;
```

```
for i=1:10
```

```
    if A(i) >0
```

```
        positivo=positivo+1;
```

```
    else
```

```
        negativo=negativo+1;
```

```
    end;
```

```
end;
```

De uma forma genérica a expressão para o index pode ser definida como:



```
for k = inicio:incremento:fim
```

ou seja k toma o valor inicial *inicio* e é incrementada de *incremento* até ao valor final *fim*.

## 1.4 Controlo de fluxo

Consideremos um exemplo. Suponhamos o mesmo problema em que a dimensão do vector A é desconhecida e pretende-se apenas contar os elementos positivos e negativos das colunas ímpares. Uma possível solução seria:



|                      |
|----------------------|
| positivo = 0;        |
| negativo = 0;        |
| for i=1:2:length(A)  |
| if A(i) > 0          |
| positivo=positivo+1; |
| else                 |
| negativo=negativo+1; |
| end;                 |
| end;                 |

### 1.4.8 Ciclo de repetição While

Ao contrário do ciclo **for**, no ciclo **while** o ciclo é executado não um número predeterminado de vezes mas enquanto uma expressão lógica for verdadeira. A sintaxe do ciclo **while** é a seguinte:



|                               |
|-------------------------------|
| <b>while expressão lógica</b> |
| <b>instruções</b>             |
| <b>end</b>                    |

Ou seja, enquanto a expressão lógica for verdadeira é executado o conjunto de instruções. Caso seja falsa é interrompido o ciclo.

O primeiro exemplo de contagem de números positivos e negativos pode ser solucionado usando um ciclo **while**:



|                      |
|----------------------|
| positivo = 0;        |
| negativo = 0;        |
| i=1;                 |
| while i<=10          |
| if A(i) >0           |
| positivo=positivo+1; |
| else;                |

|                                   |
|-----------------------------------|
| <code>negativo=negativo+1;</code> |
| <code>end;</code>                 |
| <code>i=i+1;</code>               |
| <code>end;</code>                 |

## 1.5 Gráficos 2D

### 1.5.1 Comando Plot

A maioria dos gráficos necessários são, sem dúvida, do tipo x-y. Os dados consistem normalmente num conjunto de pares ordenados de pontos e daí a designação de gráficos (x,y). Recorde-se a tabela 1.1, onde se mostram os valores observados da temperatura do ar num determinado local durante as 24 horas do dia, de duas em duas horas. Assumindo que os valores foram guardados em dois vectores



```
>> x =[ 0: 2: 22]
```

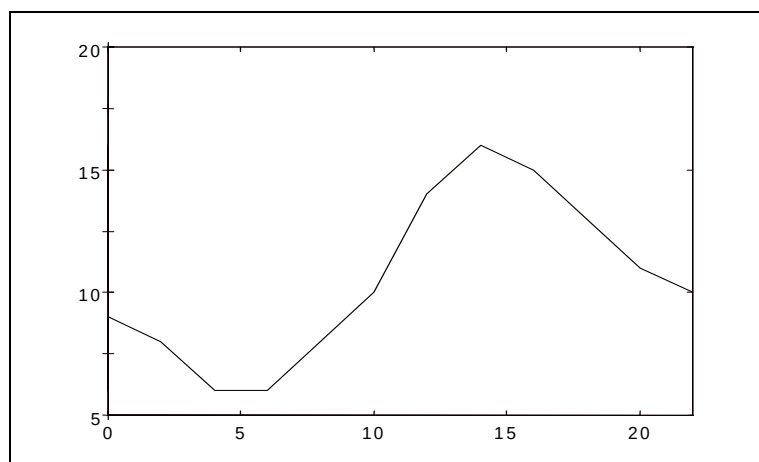
```
>> y =[ 9 8 6 5 8 10 14 17 15 13 11 10 ]
```

então, como já se referiu, o comando



```
>> plot(x,y)
```

permite visualizar os dados da seguinte forma:



**Figura 1.2:** Gráfico da temperatura do ar.

## Títulos

É ainda possível introduzir facilmente títulos, designações nos eixos e grelhas. Para isso poder-se-iam usar os seguinte comandos:

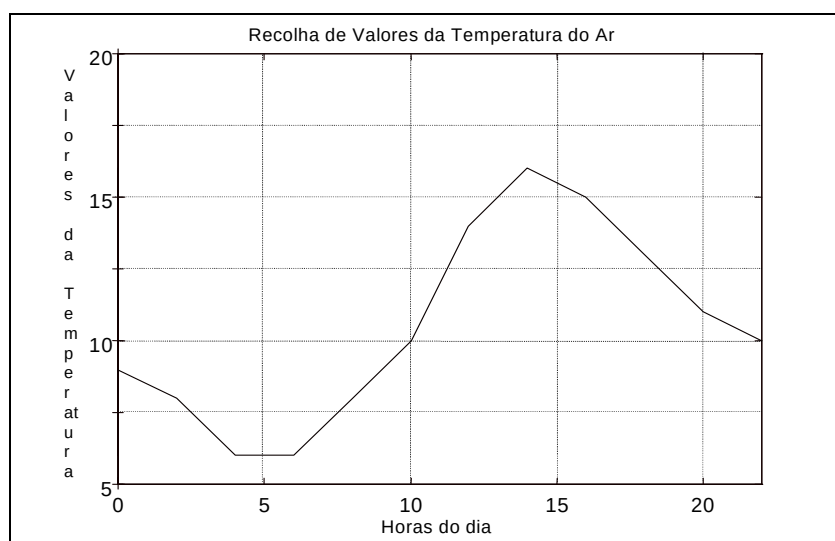


```
>> xlabel('Horas do dia')
```

```
>> ylabel('Valores da Temperatura')
```

```
>> title('Recolha de Valores da Temperatura do Ar')
```

```
>> grid
```



**Figura 1.3:** Alteração do gráfico da temperatura do ar.

### 1.5.2 Gráficos de barras

Outra possibilidade gráfica que o Matlab permite é a utilização de gráfico de barras. Existem quatro comandos,

- `bar(y)`

Desenha um gráfico de barras com os elementos do vector `y`

- `bar(x,y)`

Desenha um gráfico de barras com os elementos do vector `y` segundo as localizações específicas do vector `x` que se assume ter elementos igualmente espaçados e ordenados por ordem crescente

- `stairs(y)`

Desenha um gráfico em degraus com os elementos do vector `y`

- `stairs(x,y)`

Desenha um gráfico em degraus com os elementos do vector `y` segundo as localizações específicas do vector `x` que se assume ter

## 1.5 Gráficos 2D

elementos igualmente espaçados e ordenados por ordem crescente

A seguir mostram-se exemplos destes comandos utilizando o exemplo já conhecido da temperatura do ar.



```
>> bar(x,y)
```

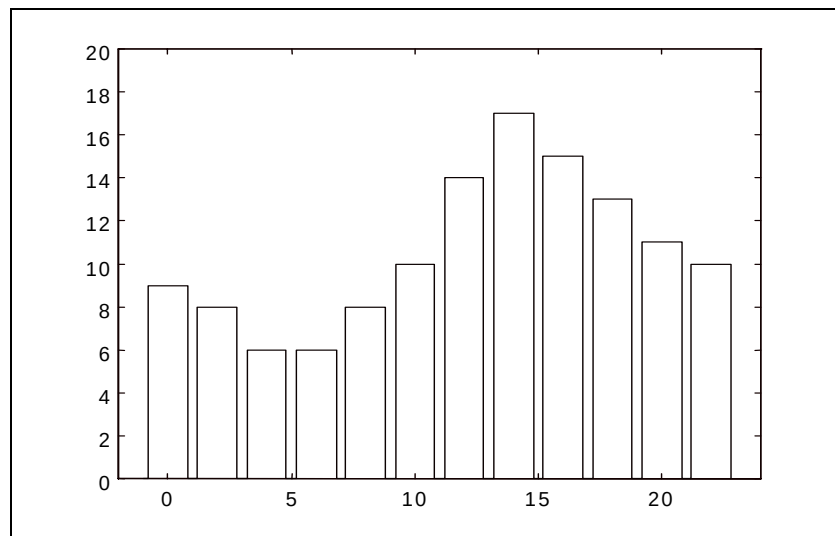


Figura 1.4: Gráfico de barras da temperatura do ar.



```
>> stairs(x,y)
```

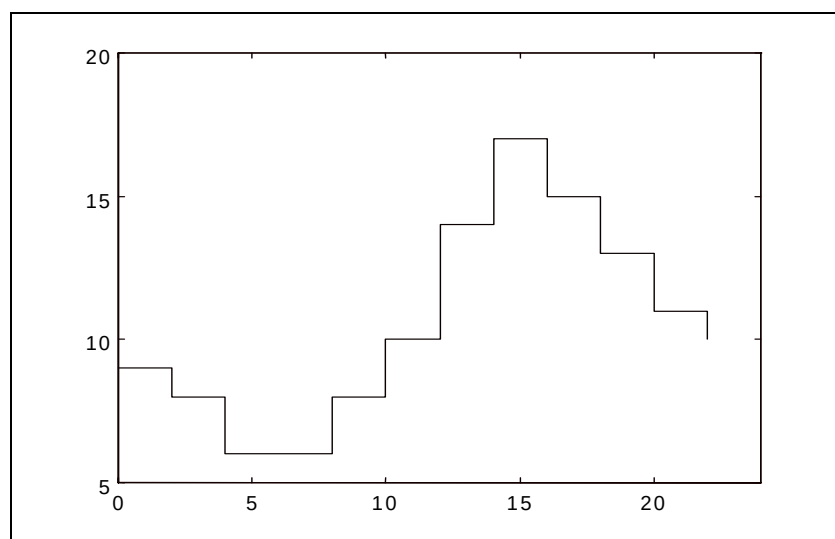


Figura 1.5: Gráfico de barras da temperatura do ar.

### 1.5.3 Opções de gráficos

Apresentam-se agora algumas opções relativas ao traçado de gráficos.

#### Representação da área da curva: fill

É possível, usando a função **fill**, representar não só a função mas também a área por ela definida numa determinada cor. Por exemplo o comando a seguir,



```
>> fill(x,y,'c')
```

em que x e y são os vectores definidos para o exemplo da temperatura do ar, permite representar o gráfico definindo um polígono de cor 'c', neste caso *cyan*.

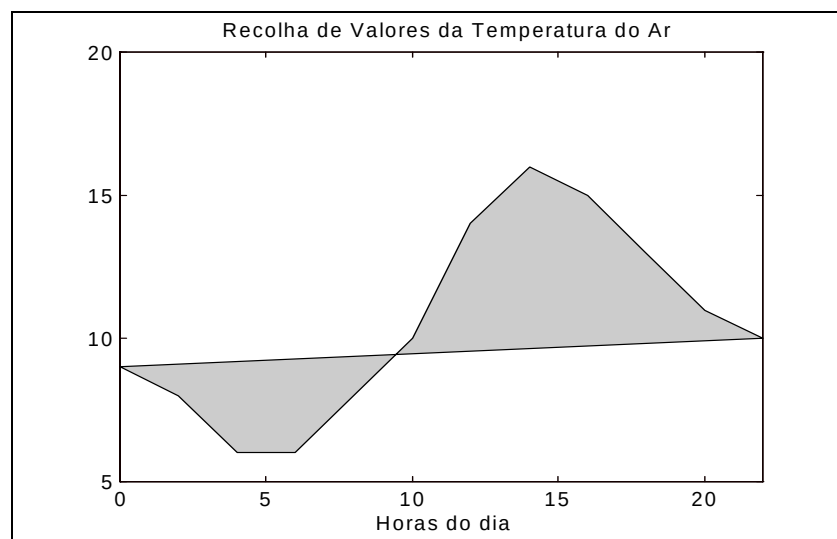


Figura 1.6: Definição de um polígono.

#### Representação simultânea de várias curvas no mesmo gráfico

Suponha que, para além da temperatura, foi recolhido também o valor da velocidade do vento durante o dia, armazenado no vector w.

| Horas | Temperatura | Vento |
|-------|-------------|-------|
| 0     | 9           | 12    |
| 2     | 8           | 13    |
| 4     | 6           | 14    |

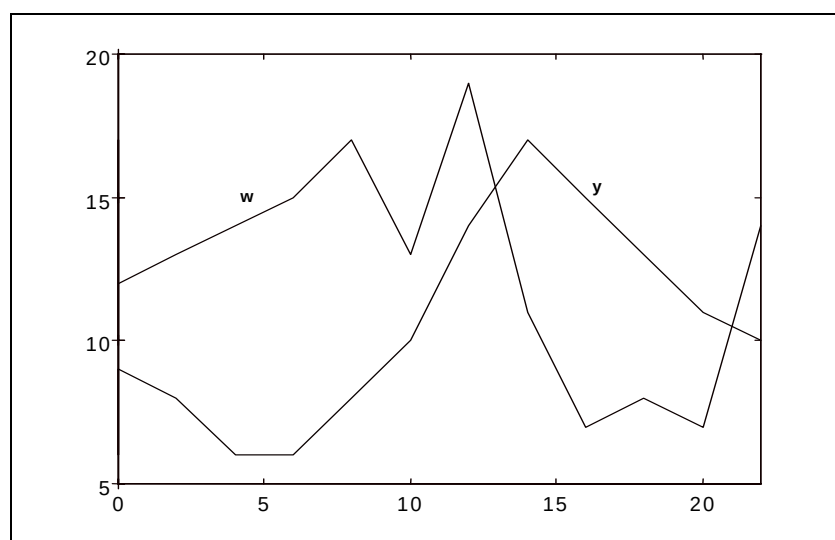
|    |    |    |
|----|----|----|
| 6  | 6  | 15 |
| 8  | 8  | 17 |
| 10 | 10 | 13 |
| 12 | 14 | 19 |
| 14 | 17 | 11 |
| 16 | 15 | 7  |
| 18 | 13 | 8  |
| 20 | 11 | 7  |
| 22 | 10 | 14 |

**Tabela 1.7:** Valores da temperatura do ar e velocidade do vento.

Deseja-se portanto representar simultaneamente a temperatura e velocidade do vento no mesmo gráfico. Em Matlab tal é possível fazendo:



```
>> plot(x,y,x,w)
```



**Figura 1.7:** Gráfico simultâneo de várias curvas.

Além de poder representar várias curvas no mesmo gráfico pode-se ainda desejar visualizar de forma diferente (para melhor as distinguir) cada uma das curvas. Em Matlab é possível representar cada curva usando dois parâmetros: tipo de linha e cor. A expressão geral é:



```
plot(x,y,'ct')
```

em que c identifica a cor e t o tipo de linha. Existem as seguintes possibilidades:

| Cor |          | Tipo |             |
|-----|----------|------|-------------|
| y   | Amarelo  | .    | Ponto       |
| m   | Mangenta | o    | Circulo     |
| c   | Cian     | +    | Símal mais  |
| r   | Vermelho | x    | Símal x     |
| g   | Verde    | -    | Sólido      |
| b   | Azul     | :    | Ponteado    |
| w   | Branco   | -.   | Traço Ponto |
| k   | Preto    | --   | Tracejado   |

Tabela 1.8: Opções de visualização.

Assim, o comando



```
>> plot(x,y, '-.',x,w, 'x')
```

teria como resultado,

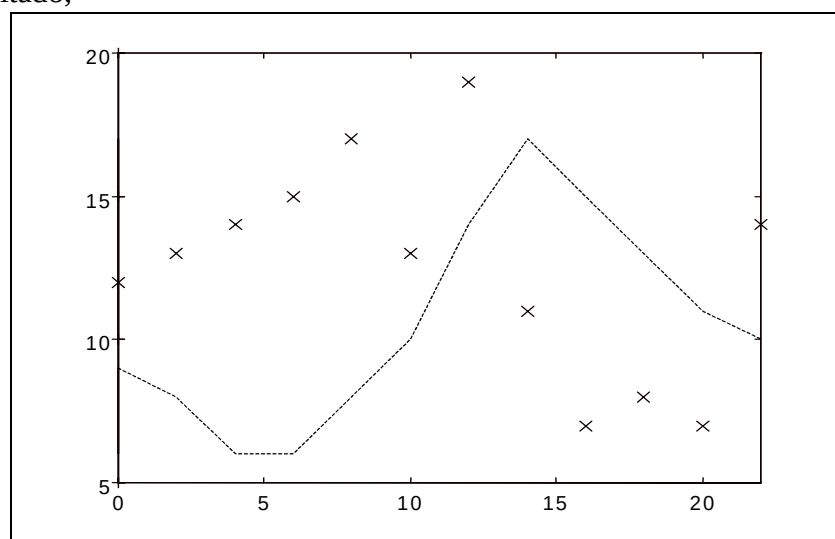


Figura 1.8: Vários tipos de curvas.

### Representação simultânea de vários gráficos

Além de por vezes se desejar visualizar várias linhas no mesmo gráfico deseja-se visualizar vários gráficos no mesmo écran. O Matlab permite-o fazer usando o comando subplot. A sintaxe deste comando é a seguinte:



```
subplot(a,b,c)
```

Os parâmetros a b e c são números inteiros que definem:

- a** - o número de linhas segundo a qual o écran vai ser dividido
- b** - o número de colunas segundo a qual o écran vai ser dividido
- c** - permite escolher qual o subécran da matriz definida por (a,b)

Um exemplo:



```
>> subplot(2,2,3)
```

Este comando permite definir o écran em duas linhas e duas colunas e seleccionar o terceiro elemento, ou seja a posição, (2,1). O próximo gráfico será visualizado apenas neste écran.

Retomando o exemplo da temperatura e velocidade do vento suponha que foram executados os seguintes comandos no Matlab,



```
>> subplot(2,2,2)
```

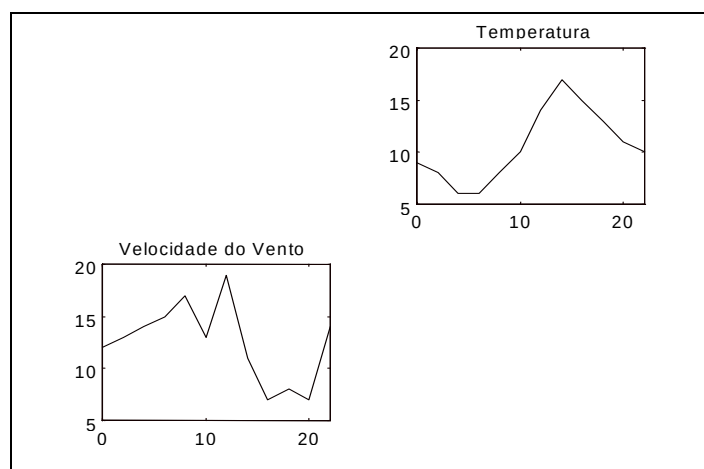
```
>> plot(x,y)
```

```
>> title(' Temperatura ')
```

```
>> subplot(2,2,3)
```

```
>> plot(x,w)
```

```
>> title(' Velocidade do Vento ')
```



**Figura 1.9:** Divisão do écran.

## 1.5 Gráficos 2D

---

Divide-se portanto o écran em duas linhas e duas colunas, selecciona-se o segundo elemento e visualiza-se a curva da temperatura, selecciona-se o terceiro elemento e visualiza-se a curva da velocidade do vento.

### Definição da Escala

O Matlab automaticamente define qual a gama de visualização para os eixos vertical e horizontal de um gráfico. É contudo possível definir a gosto do utilizador a gama e o tipo de escalas a utilizar. Para isso existe o comando **axis**. Eis algumas possibilidades:

|                         |                                                                                                    |
|-------------------------|----------------------------------------------------------------------------------------------------|
| • <b>axis(v)</b>        | Permite definir as escalas em que v é um vector de quatro elementos reais v=[ xmin xmax ymin ymax] |
| • <b>axis</b>           | Permite retornar ao valor original da escala antes de ser definida pelo comando anterior.          |
| • <b>axis('normal')</b> | É o valor por defeito.                                                                             |
| • <b>axis('square')</b> | Especificação do aspecto do gráfico como quadrado.                                                 |

### Controlo do Écran

Existem em Matlab duas janelas: de comandos e de gráficos. Na janela de comandos são inseridos os comandos e na de gráficos são visualizados os gráficos gerados por exemplo com o comando **plot**. A seguir mostram-se alguns dos comandos capazes de operar com estas duas janelas:

|                  |                                                 |
|------------------|-------------------------------------------------|
| • <b>shg</b>     | Visualizar a janela de gráficos                 |
| • qualquer tecla | Visualizar a janela de comandos                 |
| • <b>clc</b>     | Apagar a janela de comandos                     |
| • <b>clg</b>     | Apagar a janela de gráficos                     |
| • <b>home</b>    | Ir para a posição inicial da janela de comandos |

## 1.6 Gráficos 3D

Nesta secção estudam-se algumas das possibilidades do Matlab na representação de gráficos tridimensionais.

### 1.6.1 Comando Plot3

À semelhança do comando **plot** para curvas 2D existe o comando **plot3** para representar no espaço tridimensional os vectores X, Y, e Z.



```
>> plot3(x,y,z)
```

Por exemplo a seguinte sequência de comandos,



```
>> t=0:0.1:5*pi;
```

```
>> plot3( sin(t), cos(t), t)
```

```
>> xlabel('x')
```

```
>> ylabel('y')
```

```
>> zlabel('z')
```

```
>> grid
```

permite obter o seguinte gráfico:

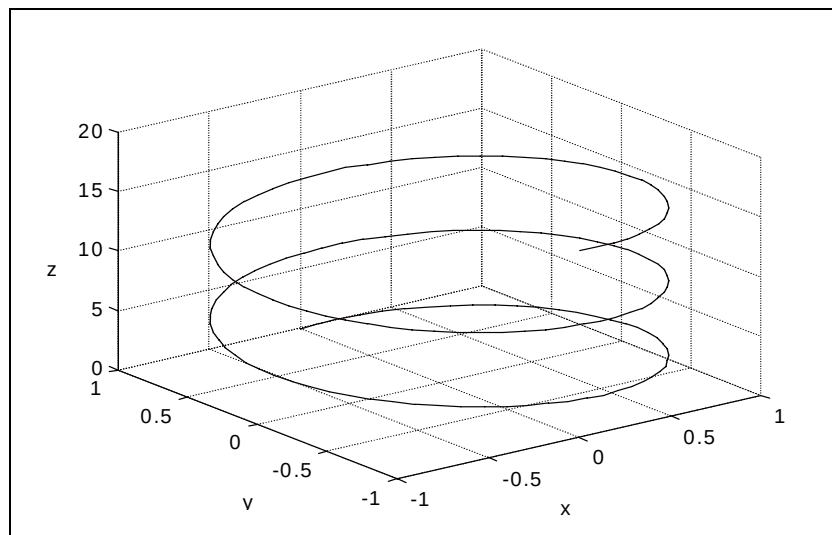


Figura 1.10: Plot 3D.

Note-se a existência do comando **zlabel** a utilizar na terceira coordenada.

### 1.6.2 Superfícies Mesh

## 1.6 Gráficos 3D

Uma superfície **mesh** é gerada a partir de um conjunto de pontos existente numa matriz: cada ponto da matriz representa o valor da superfície.

Por exemplo pretende-se gerar uma superfície tridimensional tal que as variáveis  $x$  e  $y$  são as variáveis independentes (representam o plano horizontal) e a variável  $z$  é a dependente (representa a terceira coordenada de cada ponto no espaço).

Suponhamos por exemplo que se deseja traçar o gráfico da função

$$z = f(x, y) = \sqrt{1 - x^2 - y^2}$$

ou de outra forma

$$x^2 + y^2 + z^2 = 1$$

que se sabe ser uma esfera centrada na origem e raio 1. Sabe-se ainda que  $-0.5 \leq x \leq 0.5$  e também  $-0.5 \leq y \leq 0.5$ .

Primeiro procede-se à criação da matriz  $z$  que contém valores de  $f(x,y)$ .



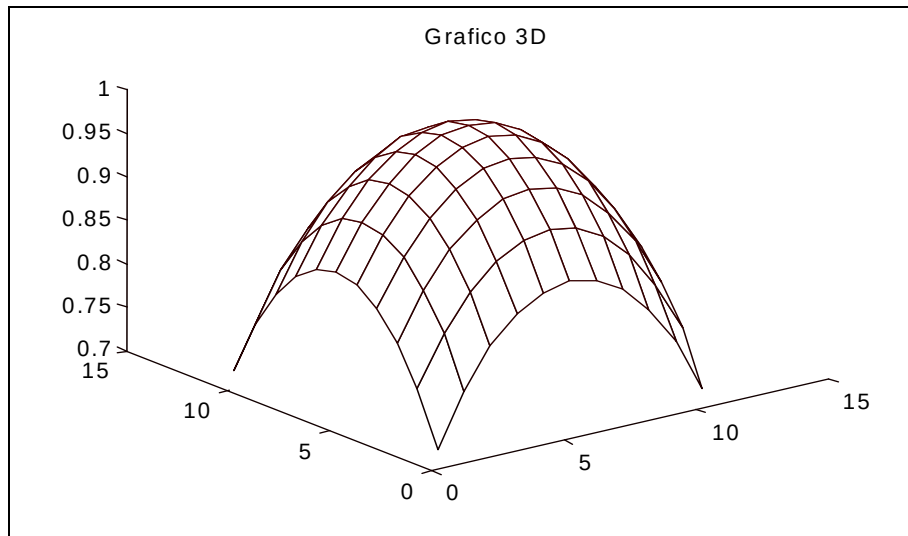
|                                |
|--------------------------------|
| i=1;                           |
| for x=-0.5:0.1:0.5;            |
| j=1;                           |
| for y=-0.5:0.1:0.5;            |
| z(i,j)=sqrt(abs(1-x.^2-y.^2)); |
| j=j+1;                         |
| end                            |
| i=i+1;                         |
| end                            |

Para traçar o gráfico basta executar,



```
>> mesh(z)

>> title (' Grafico 3D')
```



**Figura 1.11:** Gráfico 3D - função mesh.

É também possível gerar automaticamente a superfície definindo o domínio de  $x$  e  $y$  a partir do comando **meshgrid**. O mesmo exemplo poderia ser solucionado da seguinte forma:



```
>> [xdom,ydom]=meshgrid( -0.5:0.1:0.5, -0.5:0.1:0.5);
>> z= sqrt(abs(1-xdom.^2-ydom.^2));
>> mesh(z)
```

O comando **mesh(z)** visualiza a superfície sendo da responsabilidade do Matlab definir a gama de visualização. É possível alterar esta facto sendo a gama especificada pelo utilizador à custa do comando.



```
>> mesh(X,Y,z)
```

Neste caso a superfície representada seria (note-se além disso a introdução de uma grelha com o comando **grid**).

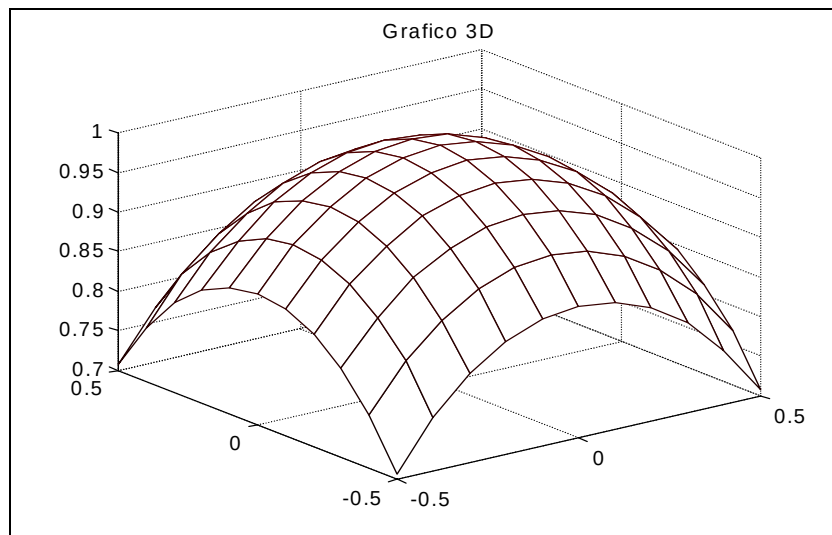


Figura 1.12: Alteração de um gráfico 3D.

### Modificação do Ângulo de Vista

Pode ser desejado visualizar uma figura segundo um ângulo diferente. A posição de vista é definida em termos de azimuth (rotação horizontal) e elevação vertical, ambas especificadas em graus. Um valor positivo de azimuth indica uma rotação no sentido contrário ao dos ponteiros do relógio. Valores positivos para a elevação vertical indicam uma rotação para cima e negativos uma rotação para baixo. O valor de defeito utilizado pelo Matlab é -37.5 para o azimuth e 30 para a elevação vertical.

É possível especificar estes parâmetros no comando **mesh**, por exemplo, o comando



```
>> mesh(z, [ -37.5, 0])
```

permite visualizar a superfície de uma elevação horizontal nula.

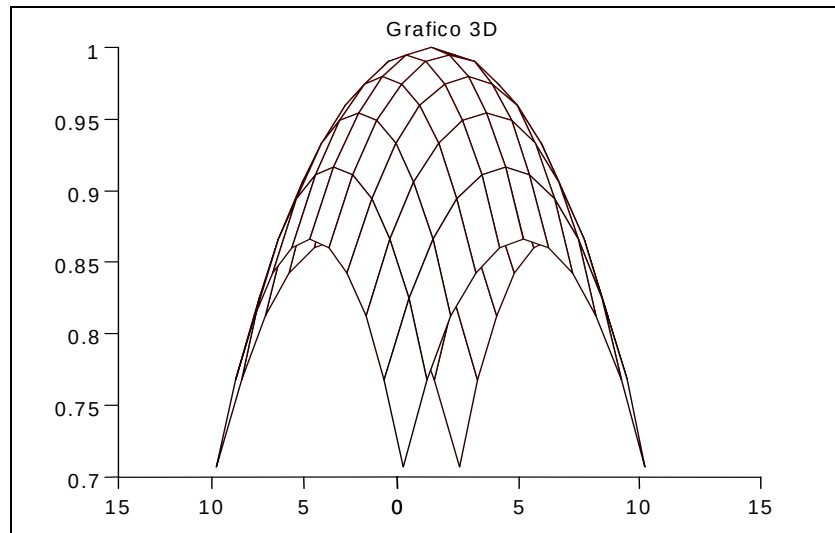
O comando,



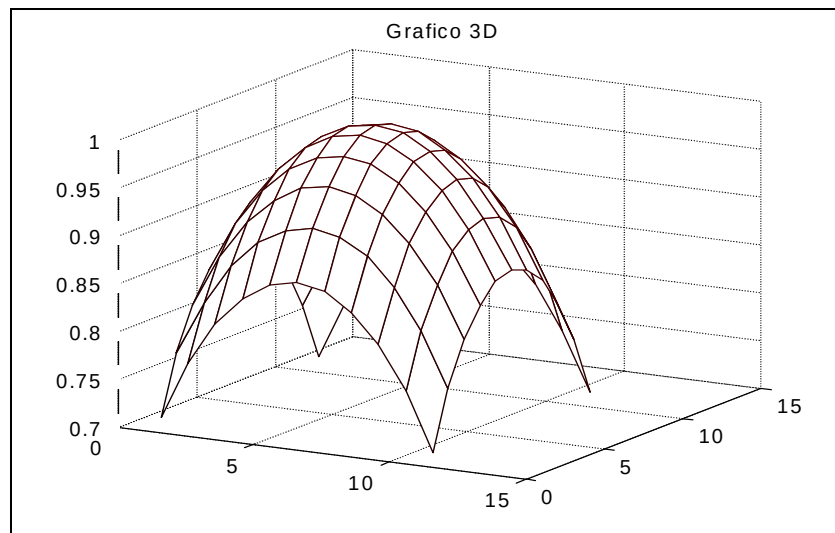
```
>> mesh(z, [ 30, 20])
```

```
>> grid
```

permite visualizar a superfície de um azimuth=30 e uma elevação horizontal=20.



**Figura 1.13:** Ângulo de vista num gráfico 3D : - 37.5, 0.



**Figura 1.14:** Azimute num gráfico 3D: 30, 20.

### 1.6.3 Superfícies Surf

Com o comando **mesh** é possível definir uma superfície tipo uma “grelha”. Usando o comando **surf** é possível representar outro tipo de superfícies, como a seguir se indica.

Suponha-se por exemplo que se deseja traçar o gráfico da função  $\sin(z)/z$ , vulgarmente conhecida por chapéu mexicano. Utilizando o comando **meshgrid** é possível determinar  $z=f(x,y)$



```
>> x=-8:0.6:8;
>> y=-8:0.6:8;
>> [X,Y] = meshgrid(x,y);
>> Z = sqrt(X.^2 + Y.^2) + eps;
>> Z = sin(Z)./Z;
>> surf(Z)
```

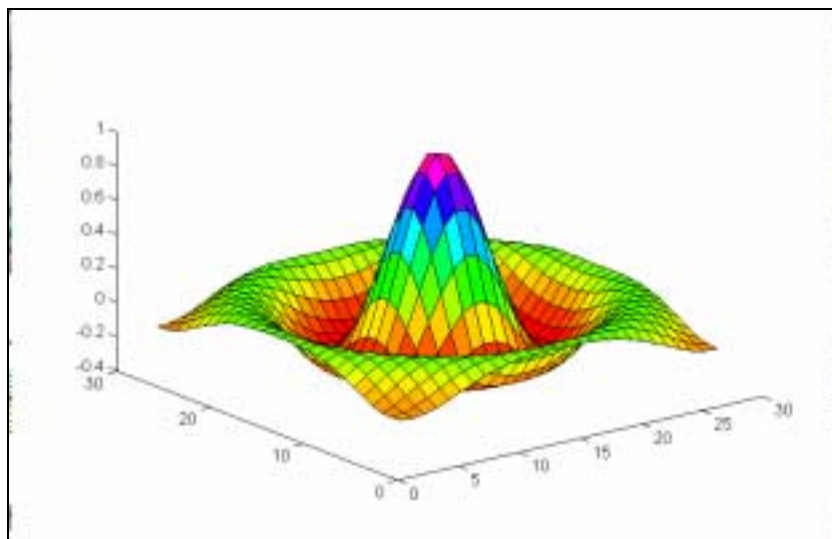


Figura 1.15: Superfície Surf - Shading Faceted.

### Modificação do Tipo de Superfície

É possível especificar uma de três tipos de superfícies possíveis:

- SHADING FACETED - valor por defeito
- SHADING FLAT - interpolação linear
- SHADING INTERP - interpolação quadrática

A seguir mostram-se tais opções fazendo uso do comando **subplot** já anteriormente explicado.



```
>> surf(X,Y,Z)
>> shading flat
>> surf(X,Y,Z)
```

```
>> shading interp
```

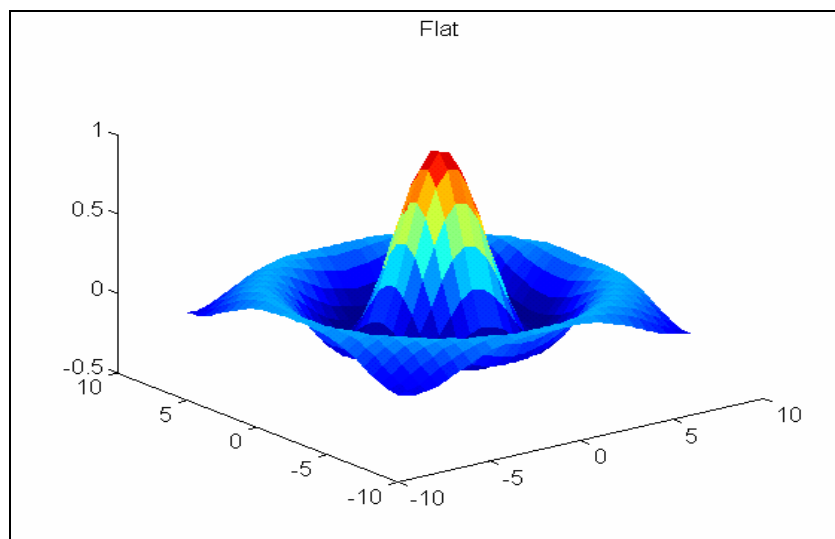


Figura 1.16: Superfície Surf - Shading Flat.

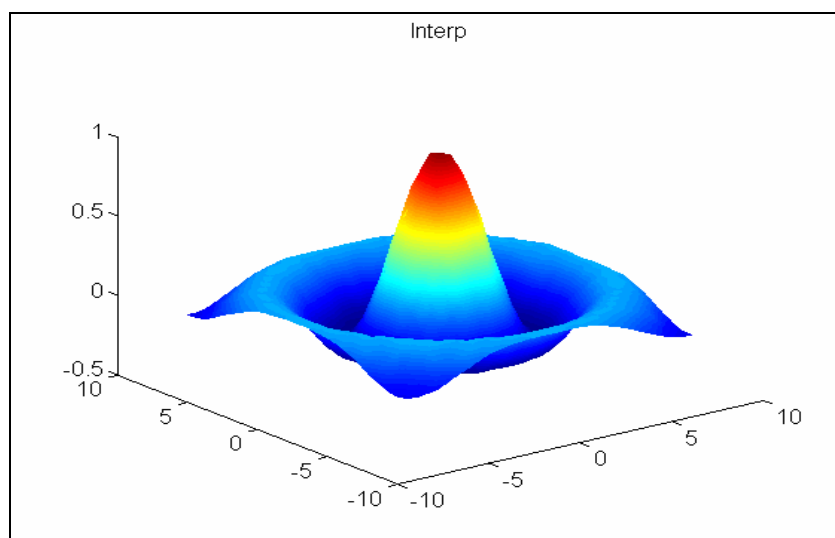


Figura 1.17: Superfície Surf - Shading Interp.

### 1.6.4 Outras Possibilidades

#### Modificação do Ângulo de Vista

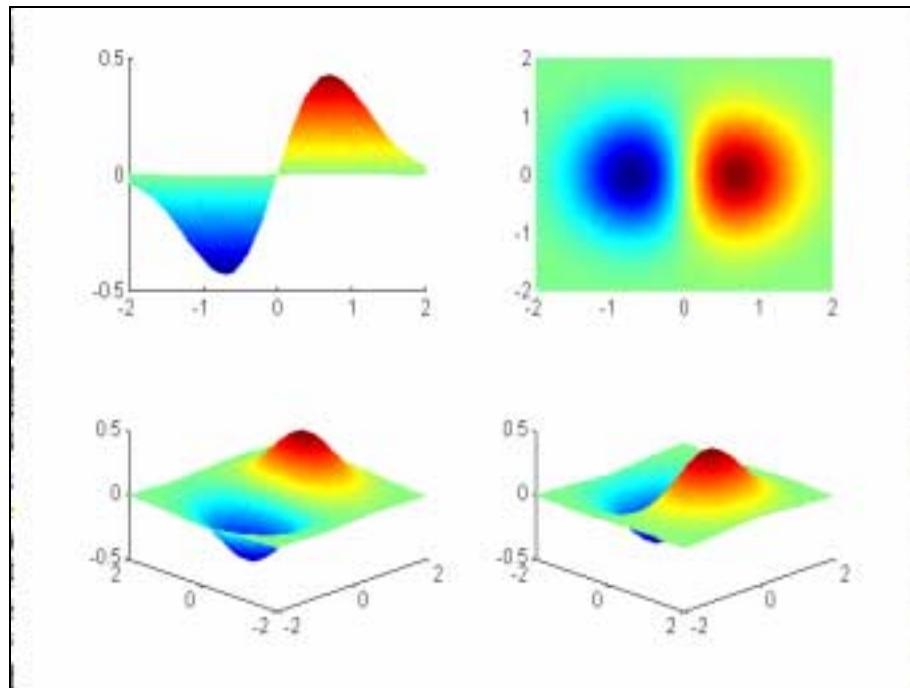
Vimos anteriormente que usando o comando **mesh** é possível definir um azimuth e uma elevação. De uma forma genérica pode ser usado o comando **view**.



```
>> view(azimute, elevacao)
```

A seguir mostra-se o gráfico da função  $z = x * e^{(-x^2-y^2)}$  usando as seguintes definições de azimuth e elevação:

- view(0,0)
- view(0,90)
- view(-45,30)
- view(45,30)



**Figura 1.18:** Ângulo de visão - comando View.

### Eliminação dos Eixos

Entre outras possibilidades já estudadas anteriormente acerca dos eixos é vulgar num gráfico tridimensional querer mostrar apenas a figura. Isso é possível com o comando,



```
>> axis('off')
```

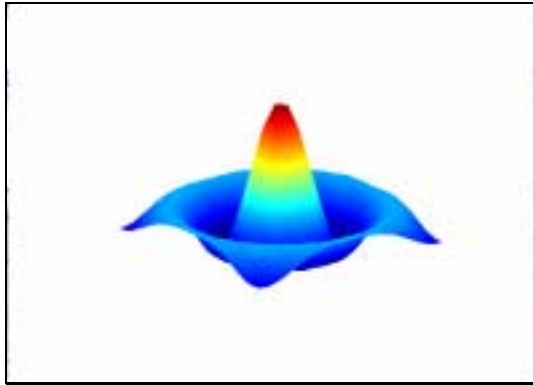
Para mostrar novamente os eixos deve-se fazer:



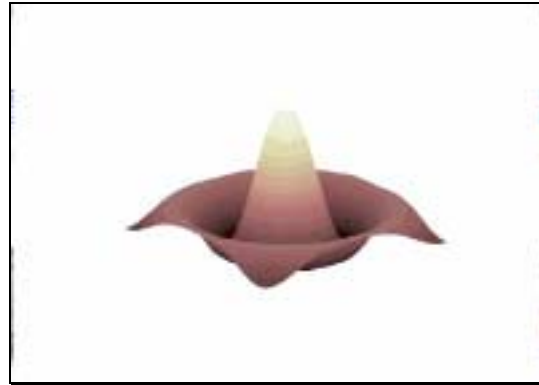
```
>> axis('on')
```

### Modificação da Cor

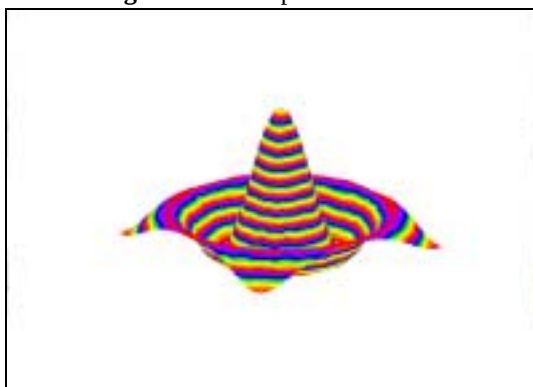
Em Matlab existe um comando, **colormap** (mapa), que permite optar por um mapa de cores. Eis algumas das possibilidades: *hsv*, *pink*, *prism* e *hot*.



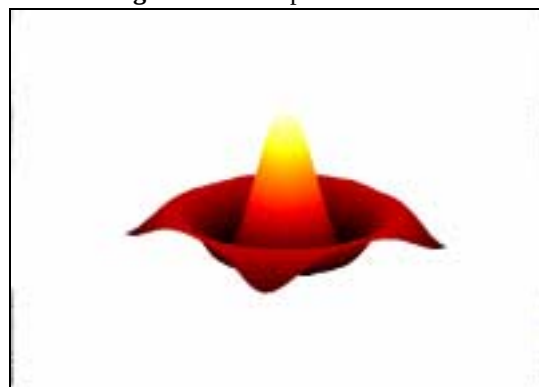
**Figura 1.19:** Mapa de cor: Hsv.



**Figura 1.20:** Mapa de cor: Pink.



**Figura 1.21:** Mapa de cor: Prism.



**Figura 1.22:** Mapa de cor: Hot.

### Eliminação de partes da figura

Além dos vectores X, Y e Z, no comando *surf* é possível especificar um quarto parâmetro, a cor da superfície. De uma forma genérica o comando será:



```
>> surf(X,Y,Z,C)
```

Por defeito, não sendo especificado o parâmetro C, assume-se que a cor é “proporcional” ao valor dos elementos da superfície, ou seja de Z. Seguindo este raciocínio é possível alterar a cor individual de cada um dos elementos da superfície.

Apesar deste assunto não ser aqui abordado, mostra-se como é possível introduzir “buracos” na superfície. Para isso os elementos a eliminar deverão ter o valor de **Nan** (variável pré definida em Matlab). Segue-se um exemplo.



```
>> C=Z;
```

```
>> xburaco=5:13;
```

```
>> yburaco=5:13;
```

```
>> C(xburaco,yburaco)=nan*C(xburaco,yburaco);
```

```
>> surf(X,Y,Z,C)
```

Não são mostrados os elementos de  $X=[5..13]$  e  $Y=[5..13]$  da superfície. Recorde-se que a dimensão de  $X$  e  $Y$  foi definida por  $X=Y=-8:0.6:8$ , portanto 27 elementos.

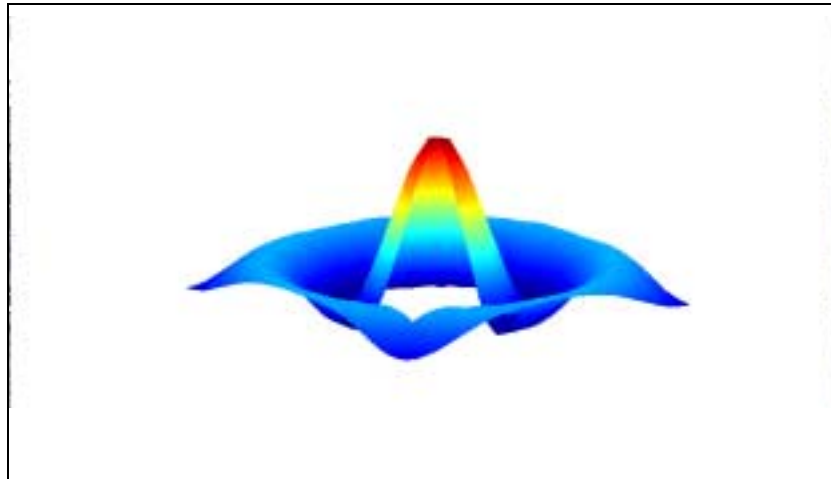


Figura 1.23: Eliminação de Partes da figura.

## 1.7 Outros Comandos e Funções

Embora não se inserindo em qualquer das secções anteriores existem outros comandos que se passam a explicar:

|                     |                                                                          |
|---------------------|--------------------------------------------------------------------------|
| • <i>who</i>        | Lista as variáveis definidas até ao momento                              |
| • <i>whos</i>       | Idêntico ao <i>who</i> só que é dada informação adicional (tamanho, ...) |
| • <i>what</i>       | Lista dos ficheiros *.m, *.mat e *.mex                                   |
| • <b>exit, quit</b> | Abandonar o Matlab                                                       |
| • <b>!</b>          | Executar um comando DOS                                                  |
| • <i>computer</i>   | Tipo de computador sobre o qual o Matlab se encontra a operar            |
| • <i>clear</i>      | Eliminar variáveis do espaço de trabalho                                 |
| • <b>^c</b>         | Abortar uma acção que está a ser executada pelo Matlab                   |
| • <i>ans</i>        | Variável usada para armazenar o resultado da última operação             |
| • <i>help</i>       | Para aceder à ajuda                                                      |

## 1.7 Outros comandos e funções

- %
- *pause*

Para inserir uma linha de comentários

Espera por uma acção do utilizador

Exemplos:



```
» help abs
```

Permite aceder à definição da função **abs** ...



```
» c=a+b      % soma de a e b
```

Permite comentar o resultado da operação a+b



```
» ! dir
```

Equivalente ao comando dir em DOS.

## 1.8 Exemplos

Apresentam-se agora alguns exemplos de aplicação em Matlab

### 1.8.1 Máximo, mínimo e média

Neste primeiro exemplo pretende-se, dada uma função  $y=f(x)$ , calcular o seu valor máximo, mínimo e médio, para um certo domínio de  $x$ . Para isso pretende-se construir uma função que permita determinar o máximo, mínimo e o valor médio de um dado conjunto de valores.

Deve-se no final fazer a representação de  $f(x)$ , assim como as rectas horizontais que definem o seu máximo, mínimo e média.

**Dados:** A função é  $f(x) = 3 * e^{-x} * \sin(5x)$  definido no intervalo  $[0,5]$ .

Função auxiliar: *mamime.m*



```
Function [ma,mi,me]=mamime(x)
```

```
Ma= max(x);
```

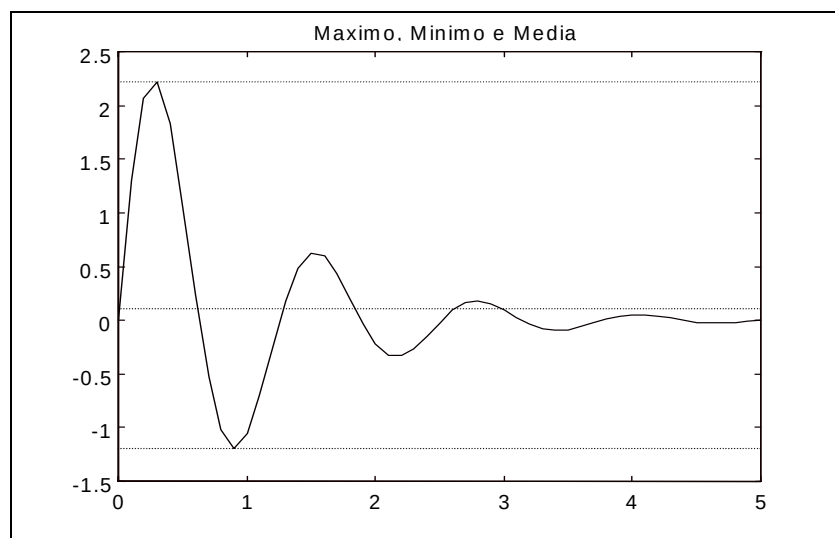
## 1.9 Índice de funções por assuntos

|              |
|--------------|
| Mi= min(x);  |
| Me= mean(x); |
| End          |

Programa principal: *exemplo1.m*



|                                                         |
|---------------------------------------------------------|
| % ----- Definicao de t e x                              |
| x=0:0.1:5;                                              |
| y=3*exp(-x).*sin(5*x);                                  |
| % ----- Chamada da funcao                               |
| [ma,mi,me]=mamime(y);                                   |
| % ----- Graficacao                                      |
| N=length(y);                                            |
| recta=ones(N,1);                                        |
| plot(x,y,x,ma*recta,':',x,mi*recta,':',x,me*recta,':'); |
| title(' Maximo, Minimo e Media')                        |



**Figura 1.24:** Máximo, mínimo e média de uma função.

### 1.8.2 Raízes de um polinómio

Pretende-se neste exemplo implementar em Matlab um programa que permita ao utilizador definir um polinómio de terceira ordem, traçar o gráfico e calcular as suas raízes. Devem ainda ser indicadas as raízes reais (pelo menos uma das raízes é real).



|                                               |
|-----------------------------------------------|
| Clear                                         |
| clc                                           |
| %----- Definicao do polinomio                 |
| disp(' INTRODUCAO DOS COEFICIENTES ')         |
| a3 = input(' Coeficiente de ordem tres : ' ); |
| a2 = input(' Coeficiente de ordem dois : ' ); |
| a1 = input(' Coeficiente de ordem um : ' );   |
| a0 = input(' Coeficiente independente : ' );  |
| polinomio=[ a3 a2 a1 a0];                     |
| %----- Definicao da gama de x                 |
| xmin = input(' Valor minimo de x ' );         |
| xmax = input(' Valor maximo de x ' );         |
| %----- Calculo de y=f(x)                      |
| x=xmin:0.01:xmax;                             |
| y= a3*x.^3 + a2*x.^2 + a1*x + a0;             |
| plot(x,y, x,0*y)                              |
| title(' Polinomio de terceira ordem ')        |
| xlabel(' x ')                                 |
| ylabel(' y ')                                 |
| %----- Calculo das raizes                     |
| raizes=roots(polinomio);                      |
| for i=1:3                                     |
| if isreal(raizes(i))                          |
| disp(' Tem uma raiz real em :')               |
| disp( raizes(i) )                             |
| pause                                         |
| end                                           |
| end                                           |

A seguir mostra-se o exemplo para o seguinte polinómio:  $f(x) = x^3 - 6x^2 - 4x + 24$ , cujas raízes são -2, 2 e 6.

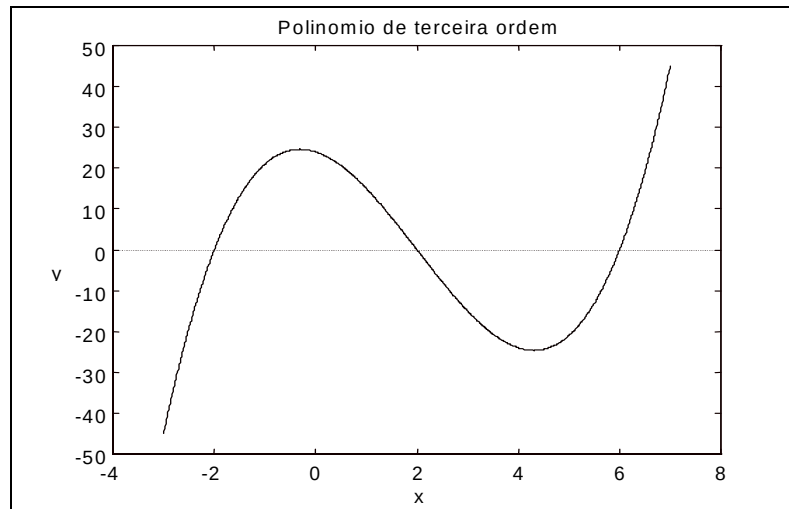


Figura 1.25: Raízes de uma função.

## 1.9 - Índice de Funções por Assuntos

Apresentam-se agora as funções mais importantes em Matlab ordenadas por assunto.

### Genéricas

| Geral           |                                |
|-----------------|--------------------------------|
| <b>help</b>     | ajuda                          |
| <b>demo</b>     | demonstração                   |
| <b>who</b>      | variáveis definidas em memória |
| <b>what</b>     | lista das *.m files            |
| <b>computer</b> | tipo computador                |
| <b>size</b>     | dimensão de uma matriz         |
| <b>length</b>   | tamanho de um vector           |
| <b>clear</b>    | eliminar uma variável          |
| <b>^c</b>       | abortar uma acção              |
| <b>quit</b>     | sair do Matlab                 |
| <b>exit</b>     | idêntica a quit                |

| Caracteres Especiais |                                  |
|----------------------|----------------------------------|
| <b>=</b>             | atribuição                       |
| <b>[</b>             | definição de vectores e matrizes |
| <b>]</b>             | ver [                            |
| <b>(</b>             | precedências em expressões       |
| <b>)</b>             | ver )                            |
| <b>.</b>             | ponto decimal                    |

|     |                                                   |
|-----|---------------------------------------------------|
| ... | continuação da instrução na próxima linha         |
| ,   | separação de elementos e argumentos de uma função |
| ;   | fim de linha, suprimir visualização               |
| %   | comentário                                        |
| :   | índices, gerar vectores                           |
| !   | executar comandos do sistema operativo DOS        |

## Matrizes

| Operadores |                    |          |                    |
|------------|--------------------|----------|--------------------|
| Matrizes   |                    | Vectores |                    |
| +          | adição             | +        | adição             |
| -          | subtracção         | -        | subtracção         |
| *          | multiplicação      | .*       | multiplicação      |
| /          | divisão à direita  | ./       | divisão à direita  |
| \          | divisão à esquerda | .\       | divisão à esquerda |
| ^          | potência           | .        | potência           |
| '          | transposta         | ^.'      | transposta         |

| Funções Matemáticas |                                                            |
|---------------------|------------------------------------------------------------|
| <b>abs</b>          | valor absoluto                                             |
| <b>sqrt</b>         | raiz quadrada                                              |
| <b>real</b>         | parte real de um número complexo                           |
| <b>imag</b>         | parte imaginária de um número complexo                     |
| <b>conj</b>         | conjugado de um número complexo                            |
| <b>round</b>        | converte para o inteiro mais próximo                       |
| <b>fix</b>          | converte para o inteiro mais próximo $\rightarrow 0$       |
| <b>floor</b>        | converte para o inteiro mais próximo $\rightarrow -\infty$ |
| <b>ceil</b>         | converte para o inteiro mais próximo $\rightarrow +\infty$ |
| <b>sign</b>         | sinal de número                                            |
| <b>rem</b>          | resto                                                      |
| <b>fmin</b>         | mínimo de uma função de uma variável                       |
| <b>fzero</b>        | zeros de uma função de uma variável                        |
| <b>fmins</b>        | mínimo de uma função de várias variáveis                   |
| <b>fzeros</b>       | zeros de uma função de várias variáveis                    |
| <b>sin</b>          | seno                                                       |

|              |                            |
|--------------|----------------------------|
| <b>cos</b>   | coseno                     |
| <b>tan</b>   | tangente                   |
| <b>asin</b>  | arco seno                  |
| <b>acos</b>  | arco coseno                |
| <b>atan</b>  | arco tangente              |
| <b>sinh</b>  | seno hiperbólico           |
| <b>cosh</b>  | coseno hiperbólico         |
| <b>tanh</b>  | tangente hiperbólico       |
| <b>exp</b>   | exponencial (base e)       |
| <b>log</b>   | logaritmo natural (base e) |
| <b>log10</b> | logaritmo (base 10)        |

Matrizes...

| Definição de Matrizes |                              |
|-----------------------|------------------------------|
| <b>diag</b>           | matriz diagonal              |
| <b>eye</b>            | matriz identidade            |
| <b>ones</b>           | matriz de uns                |
| <b>zeros</b>          | matriz de zeros              |
| <b>rand</b>           | matriz aleatória             |
| <b>magic</b>          | matriz mágica                |
| <b>tril</b>           | matriz triangular inferior   |
| <b>triu</b>           | matriz triangular superior   |
| <b>det</b>            | determinante de uma matriz   |
| <b>rank</b>           | característica de uma matriz |

| Operações por colunas/linhas (vectors) |                                      |
|----------------------------------------|--------------------------------------|
| <b>max</b>                             | máximo                               |
| <b>min</b>                             | mínimo                               |
| <b>mean</b>                            | valor médio                          |
| <b>std</b>                             | desvio padrão                        |
| <b>median</b>                          | mediana                              |
| <b>sort</b>                            | ordenação                            |
| <b>sum</b>                             | soma dos elementos                   |
| <b>prod</b>                            | produto dos elementos                |
| <b>diff</b>                            | diferenças (aproximação da derivada) |
| <b>hist</b>                            | histograma                           |
| <b>table1</b>                          | interpolação linear                  |
| <b>corr</b>                            | matriz de correlação                 |
| <b>cov</b>                             | matriz de covariância                |
| <b>spline</b>                          | interpolação cúbica                  |

| Valores Especiais |                                                        |
|-------------------|--------------------------------------------------------|
| <b>ans</b>        | resposta de uma expressão não atribuída a uma variável |
| <b>eps</b>        | precisão                                               |
| <b>pi</b>         | $\pi = 3.14159...$                                     |
| <b>inf</b>        | $\infty$                                               |
| <b>Nan</b>        | indica que não é um número                             |
| <b>clock</b>      | relógio                                                |
| <b>nargin</b>     | número de argumentos de entrada numa função            |
| <b>nargout</b>    | número de argumentos de saída numa função              |

### Controlo Fluxo / Lógica

| Controlo de Fluxo |                                              |
|-------------------|----------------------------------------------|
| <b>if</b>         | expressão condicional                        |
| <b>elseif</b>     | usada com if                                 |
| <b>else</b>       | usada com if                                 |
| <b>end</b>        | fim de if, for, while                        |
| <b>for</b>        | repetição um número conhecido de vezes       |
| <b>while</b>      | repetição enquanto se verificar uma condição |
| <b>break</b>      | terminar ciclos for ou while                 |
| <b>return</b>     | retornar de funções                          |
| <b>pause</b>      | pausa até carregar numa tecla                |

| Operadores Lógicos e Relacionais |                |              |     |
|----------------------------------|----------------|--------------|-----|
| <b>&lt;</b>                      | menor          | <b>&amp;</b> | AND |
| <b>&lt;=</b>                     | menor ou igual | <b> </b>     | OR  |
| <b>&gt;</b>                      | maior          | <b>~</b>     | NOT |
| <b>&gt;=</b>                     | maior ou igual |              |     |
| <b>==</b>                        | igual          |              |     |
| <b>~=</b>                        | diferente      |              |     |

| Funções Lógicas |                                       |
|-----------------|---------------------------------------|
| <b>any</b>      | condição lógica                       |
| <b>all</b>      | condição lógica                       |
| <b>find</b>     | procura de índices de valores lógicos |
| <b>isnan</b>    | detectar Nan                          |
| <b>finit</b>    | detectar infinitos                    |
| <b>isempty</b>  | detectar matrizes vazias              |

## 1.9 Índice de funções por assuntos

---

|              |                                 |
|--------------|---------------------------------|
| <b>exist</b> | se existe definida uma variável |
| <b>isstr</b> | se é uma string                 |

**Gráficos**

| <b>Gráfica</b>  |                               | <b>Comando</b> |                                   |
|-----------------|-------------------------------|----------------|-----------------------------------|
| <b>plot</b>     | gráfico linear x-y            | <b>format</b>  | formato de números                |
| <b>polar</b>    | gráfico polar                 | <b>disp</b>    | visualização de texto ou matrizes |
| <b>mesh</b>     | superfície 3D                 | <b>fprintf</b> | visualização formatada de números |
| <b>meshgrid</b> | domínio em superfícies 3D     | <b>clc</b>     | apagar janela de comando          |
| <b>bar</b>      | gráficos de barras            | <b>home</b>    | posição superior esquerda         |
| <b>grid</b>     | grelha                        |                |                                   |
| <b>title</b>    | título em gráficos            |                |                                   |
| <b>xlabel</b>   | eixo dos x                    |                |                                   |
| <b>ylabel</b>   | eixo dos y                    |                |                                   |
| <b>text</b>     | texto                         |                |                                   |
| <b>axis</b>     | definição de eixos            |                |                                   |
| <b>hold</b>     | manter um gráfico             |                |                                   |
| <b>shg</b>      | mostrar a janela dos gráficos |                |                                   |
| <b>clg</b>      | apagar a janela dos gráficos  |                |                                   |
| <b>subplot</b>  | divisão da janela de gráficos |                |                                   |
| <b>print</b>    | impressão de gráficos         |                |                                   |
| <b>surf</b>     | superfície 2D                 |                |                                   |
| <b>plot3</b>    | curva 3D                      |                |                                   |
| <b>view</b>     | ângulo de vista               |                |                                   |
| <b>shading</b>  | tipo superfície               |                |                                   |
| <b>colormap</b> | mapa de cores                 |                |                                   |
| <b>caxis</b>    | modificação escala de cores   |                |                                   |

## Outros

| Texto e Strings |                                     |
|-----------------|-------------------------------------|
| <b>abs</b>      | converter strings em valores ASCII  |
| <b>num2str</b>  | converter números em strings        |
| <b>int2str</b>  | converter inteiros para strings     |
| <b>setstr</b>   | indicar que uma matriz é uma string |
| <b>sprintf</b>  | converter números para stringd      |

| Polinómios     |                                   |
|----------------|-----------------------------------|
| <b>poly</b>    | polinómio característico          |
| <b>roots</b>   | raízes de um polinómio            |
| <b>polyval</b> | cálculo do valor de um polinómio  |
| <b>conv</b>    | multiplicação de polinómios       |
| <b>deconv</b>  | divisão de polinómios             |
| <b>residue</b> | expansão em fracções parciais     |
| <b>polyfit</b> | ajuste de curvas por um polinómio |

| Gestão de Ficheiros |                               |
|---------------------|-------------------------------|
| <b>chdir</b>        | mudança de directoria         |
| <b>delete</b>       | eliminar ficheiros            |
| <b>dir</b>          | lista de uma directoria       |
| <b>load</b>         | carregar um ficheiro de disco |
| <b>save</b>         | guardar um ficheiro em disco  |
| <b>type</b>         | visualização de um ficheiro   |
| <b>what</b>         | lista de ficheiros *.m        |